



Faculdades de Campinas

ENGENHARIA DA COMPUTAÇÃO

Tratamento e Armazenamento de Informação

GRUPO:

Felipe Furlan RA 202210339

Henrique Biruel RA 202210347

Henrique Palorca RA 202210348

Campinas

Setembro de 2026

1 INTRODUÇÃO

O presente trabalho apresenta as respostas às dez questões dissertativas propostas na aula sobre bancos de dados NoSQL da disciplina Tratamento e Armazenamento da Informação. O material de referência organiza o estudo em torno de um ciclo completo de projeto — requisitos, modelagem, criação, armazenamento, hospedagem e monitoramento — e percorre as principais famílias de bancos não relacionais: documentos, chave-valor, wide-column, grafos, séries temporais, busca e vetores, além das plataformas multi-modelo (MARCUSO, 2026). A discussão desenvolvida a seguir procura articular os conceitos teóricos de sistemas distribuídos, especialmente o teorema CAP e os diferentes níveis de consistência, com decisões práticas de modelagem e de operação, sempre tomando como fio condutor a ideia de que a escolha tecnológica deve derivar do padrão de acesso e do problema de negócio, e não de preferências arbitrárias.

2 QUESTÃO 1 — MODELAGEM ORIENTADA AOS PADRÕES DE CONSULTA

Explique por que a modelagem NoSQL costuma ser orientada aos padrões de consulta.

Na modelagem relacional clássica, o ponto de partida é a estrutura lógica dos dados: identificam-se entidades, relacionamentos e dependências funcionais e aplica-se a normalização para eliminar redundância e anomalias de atualização (ELMASRI; NAVATHE, 2011). As consultas são tratadas posteriormente, pois o otimizador do sistema gerenciador é capaz de combinar tabelas por meio de junções em tempo de execução. Em bancos NoSQL essa ordem se inverte. Como a maioria desses sistemas foi projetada para operar de forma distribuída e não oferece junções eficientes entre partições — quando as oferece —, a pergunta que orienta o projeto deixa de ser “como normalizar?” e passa a ser “como vou consultar?” (MARCUSO, 2026). Essa abordagem é usualmente chamada de *query-first*.

A consequência prática é que o modelo de dados é desenhado para responder a um conjunto conhecido de consultas críticas, com seus filtros, ordenações, cardinalidades e volumes previstos. Sadalage e Fowler (2012) descrevem esse deslocamento por meio do conceito de agregado: em vez de

espalhar uma entidade por diversas tabelas, agrupam-se em uma única unidade de armazenamento os dados que são lidos e escritos em conjunto. A denormalização deixa de ser um desvio e torna-se decisão consciente de projeto, com um custo explícito: a duplicação de informação transfere para a aplicação a responsabilidade de manter a coerência entre cópias, o que pressupõe tolerância a janelas de consistência eventual (KLEPPMANN, 2017). Somam-se a isso decisões que também dependem do padrão de acesso, como a escolha da chave de partição, a criação de índices seletivos — que aceleram leituras, mas oneram escrita, memória e armazenamento — e a definição de políticas de retenção e *TTL* desde a fase de requisitos.

Um exemplo concreto ilustra o raciocínio. Em um catálogo de comércio eletrônico modelado em MongoDB, a página de produto costuma ser representada por um único documento que já embute variações, atributos e um resumo das avaliações, de modo que a renderização exija uma única leitura por chave primária. Em Cassandra, o mesmo princípio é levado ao extremo: cria-se uma tabela por padrão de consulta, ainda que isso implique gravar o mesmo dado em estruturas diferentes. Em ambos os casos, a estrutura existe em função da consulta, e não o contrário.

3 QUESTÃO 2 — BANCOS DE DOCUMENTOS E BANCOS CHAVE-VALOR

Compare document e key-value em estrutura, consulta e casos de uso.

Bancos chave-valor e bancos de documentos pertencem à mesma linhagem conceitual — ambos organizam a informação em agregados acessados por uma chave —, mas se distinguem pelo grau de conhecimento que o banco possui sobre o conteúdo armazenado (SADALAGE; FOWLER, 2012). No modelo chave-valor, o valor é opaco: trata-se de um *blob* que o banco armazena e devolve sem interpretar. No modelo de documentos, o valor é uma estrutura autodescritiva, tipicamente JSON ou BSON, cujos campos internos o banco compreende, indexa e consulta.

Essa diferença determina a forma de consulta e, por consequência, os casos de uso. Em um banco chave-valor, praticamente todo acesso ocorre pela chave, o que exige que ela seja projetada como elemento central do modelo; consultas por atributo interno só são possíveis mediante estruturas auxiliares construídas

manualmente pela aplicação, como índices secundários emulados por conjuntos. Em compensação, o caminho de acesso é curto e previsível, o que resulta em latências muito baixas e em escalabilidade horizontal simples. Bancos de documentos aceitam consultas por campos aninhados, filtros compostos, agregações e índices secundários, ao custo de maior complexidade interna e de latências tipicamente superiores às de um repositório em memória. O Quadro 1 sintetiza a comparação.

Quadro 1 — Comparação entre bancos de documentos e bancos chave-valor

Critério	Bancos de documentos	Bancos chave-valor
Estrutura	Documento autodescritivo (JSON/BSON) com esquema flexível e campos aninhados	Par chave→valor, com valor opaco para o banco
Consulta	Por chave, por campos internos, filtros compostos, agregações e índices secundários	Predominantemente por chave; acesso por atributo exige estrutura auxiliar na aplicação
Casos de uso	Catálogos, perfis, CMS, APIs e conteúdo de formato variável	Cache, sessão, carrinho, rate limit, ranking e filas leves
Exemplos de mercado	MongoDB, Couchbase, Amazon DocumentDB, Azure Cosmos DB	Redis, Valkey, Amazon DynamoDB, Aerospike, Memcached
Ponto crítico	Definir o agregado e evitar documentos que crescem sem limite	Projetar a chave e a política de expiração dos dados

Fonte: elaborado pelo autor com base em Sadalage e Fowler (2012) e em Marcusso (2026).

Na prática, os dois modelos frequentemente convivem. Em uma loja virtual, o catálogo pode residir em um banco de documentos, porque cada categoria possui atributos distintos e o conteúdo evolui com frequência, enquanto o carrinho e a sessão do usuário permanecem em Redis, com expiração automática e tempo de resposta na ordem de poucos milissegundos. A escolha, novamente, decorre do padrão de acesso: leitura rica e variável de um lado, leitura e escrita triviais e intensas do outro.

4 QUESTÃO 3 — EFEITOS DE UMA PARTITION KEY INADEQUADA

Mostre como uma partition key inadequada prejudica Cassandra ou DynamoDB.

Em bancos wide-column distribuídos, a *partition key* é o elemento que determina em qual nó do cluster um dado será fisicamente armazenado. Tanto no Apache Cassandra quanto no Amazon DynamoDB, o valor da chave passa por uma

função de dispersão cujo resultado indica a partição responsável, de modo que a distribuição de carga entre os nós é consequência direta da distribuição dos valores dessa chave (KLEPPMANN, 2017). Uma chave mal escolhida, portanto, não é apenas um detalhe de modelagem: ela compromete a premissa de escalabilidade horizontal sobre a qual esses sistemas se sustentam.

O problema mais recorrente é o *hotspot*. Quando a chave possui baixa cardinalidade ou concentra valores em determinado intervalo, um subconjunto pequeno de partições recebe a maior parte das requisições. Suponha-se um sistema de telemetria que utilize a data corrente como chave de partição: durante todo o dia, cada leitura e cada escrita recai sobre a mesma partição, enquanto o restante do cluster permanece ocioso. Em Cassandra, o efeito é duplo, porque a partição também cresce indefinidamente — uma *wide partition* com milhões de linhas degrada a leitura, aumenta a pressão sobre a memória, torna a compactação mais custosa e pode inviabilizar a reparação de réplicas. Em DynamoDB, a capacidade de leitura e escrita é distribuída entre partições; com uma chave concentradora, a aplicação sofre *throttling* mesmo quando a capacidade agregada provisionada seria suficiente, e o custo cresce sem ganho de desempenho correspondente.

A correção passa por aumentar a cardinalidade e distribuir a carga. No exemplo citado, a solução usual é compor a chave com o identificador do dispositivo e um intervalo temporal — técnica conhecida como *bucketing* —, resultando em algo como a combinação de identificador e dia, com o instante da medição atuando como chave de agrupamento. Quando ainda assim persiste concentração, aplica-se *write sharding*, acrescentando à chave um sufixo calculado que espalha as escritas por várias partições, ao custo de exigir consultas paralelas na leitura. Esse conjunto de decisões evidencia por que a definição da chave de partição deve anteceder qualquer implementação.

5 QUESTÃO 4 — QUANDO O MODELO EM GRAFO SUPERA AS JUNÇÕES RELACIONAIS

Explique quando um grafo é superior a joins relacionais.

Bancos de grafos representam explicitamente nós e relações, tratando a relação como cidadã de primeira classe, dotada de tipo, direção e propriedades. A

vantagem decisiva desse modelo aparece no custo de percorrer relacionamentos: em implementações baseadas em adjacência sem índice, cada nó mantém referências diretas aos seus vizinhos, de modo que a travessia depende do tamanho da vizinhança visitada e não do tamanho total da base (ROBINSON; WEBBER; EIFREM, 2015). Em um banco relacional, o mesmo percurso exige uma junção por nível de profundidade, e cada junção adicional multiplica o custo, ainda que existam índices adequados (SILBERSCHATZ; KORTH; SUDARSHAN, 2012).

Disso decorre um critério prático de escolha. O modelo em grafo se mostra superior quando a pergunta de negócio é sobre o caminho, e não sobre o conjunto: existe alguma ligação entre estas duas contas, e por quantos intermediários; quais produtos foram comprados por pessoas semelhantes a esta; qual a cadeia de fornecedores afetada pela falha de um componente; quem, em uma estrutura de permissões, herda acesso a determinado recurso. São consultas de profundidade variável, muitas vezes desconhecida em tempo de projeto, que em SQL exigiriam recursão e produziram planos de execução instáveis. O ganho é ainda maior quando os relacionamentos são densos e heterogêneos, situação em que o esquema relacional tenderia a acumular tabelas associativas.

Há, contudo, limites que precisam ser reconhecidos. Grafos não substituem o modelo relacional em cargas transacionais tabulares, com forte integridade referencial e alto volume de escritas simples, nem competem com bancos colunares em consultas analíticas agregadas sobre grandes volumes. Um caso ilustrativo é o de antifraude em serviços financeiros: o cadastro de clientes e o razão contábil permanecem em um banco relacional, enquanto o grafo armazena as ligações entre contas, dispositivos, endereços e beneficiários, permitindo identificar anéis de fraude por meio de consultas de caminho de comprimento variável. Trata-se, portanto, de complementaridade, e não de substituição.

6 QUESTÃO 5 — INFLUXDB E TIMESCALEDB: COMPARAÇÃO CONCEITUAL

Compare InfluxDB e TimescaleDB do ponto de vista conceitual.

InfluxDB e TimescaleDB atendem à mesma classe de problema — o armazenamento e a consulta de séries temporais, em que o tempo é a dimensão organizadora dos dados —, mas partem de premissas arquiteturais opostas. O

InfluxDB é um banco construído especificamente para esse fim: seu modelo de dados é formado por *measurement*, *tags* indexadas, *fields* com os valores medidos e um *timestamp*, e seu mecanismo de armazenamento foi projetado para ingestão intensa, compressão elevada e descarte automático por política de retenção. O TimescaleDB, por sua vez, é uma extensão do PostgreSQL: as séries são armazenadas em *hypertables*, tabelas particionadas de forma transparente em fragmentos por intervalo de tempo, preservando integralmente o dialeto SQL, os tipos, os índices e o ecossistema do banco relacional hospedeiro.

Quadro 2 — Comparação conceitual entre InfluxDB e TimescaleDB

Critério	InfluxDB	TimescaleDB
Natureza	Banco especializado em séries temporais, com mecanismo próprio	Extensão do PostgreSQL, com particionamento automático por tempo
Modelo de dados	Measurement, tags, fields e timestamp	Tabelas relacionais convencionais organizadas em hypertables
Linguagem	Linguagens próprias do produto, orientadas a séries	SQL completo, incluindo junções com dados relacionais
Integração	Forte com pilhas de observabilidade e coleta de métricas	Forte com o ecossistema PostgreSQL, ferramentas e bibliotecas existentes
Ponto forte	Ingestão e retenção de métricas com baixo esforço de configuração	Combinar séries temporais e dados de negócio no mesmo banco
Ponto de atenção	Ecossistema e linguagem específicos, com curva de aprendizagem própria	Herda as características operacionais e os limites do PostgreSQL

Fonte: elaborado pelo autor com base em Marcusso (2026) e na documentação oficial dos produtos.

Do ponto de vista conceitual, a escolha depende de quanto os dados temporais precisam conversar com dados de negócio e de qual competência a equipe já possui. Quando a carga é essencialmente de observabilidade — métricas de infraestrutura coletadas por agentes e visualizadas em painéis —, o modelo especializado tende a exigir menos esforço de modelagem e de operação. Quando as medições precisam ser cruzadas com cadastros, contratos ou ordens de serviço, a possibilidade de executar junções em SQL sobre a mesma instância elimina uma integração inteira. Um cenário de indústria 4.0 evidencia esse critério: o cadastro de ativos e as ordens de manutenção residem naturalmente em um banco relacional, e manter as leituras dos sensores em uma hypertable no mesmo PostgreSQL simplifica consultas que combinam desempenho do equipamento e histórico de intervenções.

7 QUESTÃO 6 — ÍNDICE INVERTIDO EM ELASTICSEARCH E OPENSEARCH

Explique índice invertido e seu papel em Elasticsearch/OpenSearch.

O índice invertido é a estrutura que sustenta os mecanismos de busca textual. Enquanto uma tabela convencional associa um identificador ao seu conteúdo, o índice invertido faz o caminho oposto: associa cada termo do vocabulário à lista de documentos em que ele ocorre, geralmente acompanhada da frequência e das posições em que aparece. Consultar passa a ser, então, uma operação de recuperação e interseção de listas de ocorrências, e não uma varredura sequencial da base. É essa inversão que torna viável pesquisar uma expressão em milhões de documentos em poucos milissegundos, algo inatingível por meio de comparações de subcadeia em um banco relacional (REDMOND; WILSON, 2012).

A construção do índice não é imediata: o texto passa por um processo de análise que envolve tokenização, normalização de caixa e de acentuação, remoção de termos irrelevantes, redução a radicais e eventual expansão por sinônimos. As decisões tomadas nessa etapa determinam o que será encontrável, razão pela qual alterar um analisador implica reindexar o conteúdo. Sobre o índice atuam ainda os algoritmos de pontuação, que ordenam os resultados por relevância em função da frequência do termo no documento e de sua raridade no conjunto. Em Elasticsearch e OpenSearch, essa estrutura é distribuída em fragmentos e replicada entre nós, o que agrega escalabilidade horizontal e disponibilidade à capacidade de busca.

Os custos são igualmente relevantes para o projeto. A indexação consome processamento e espaço adicional, a atualização não é instantânea — os documentos tornam-se pesquisáveis após um intervalo de atualização, caracterizando um comportamento de tempo quase real — e o consumo de memória cresce com o número de campos e de fragmentos. Por essas razões, a boa prática recomenda tratar o mecanismo de busca como índice derivado, e não como fonte da verdade: em um catálogo de comércio eletrônico, o produto permanece no banco transacional e é projetado para o índice de busca, que responde por pesquisa textual, filtros facetados e ordenação por relevância.

8 QUESTÃO 7 — CICLO DE VIDA DOS EMBEDDINGS EM UM BANCO VETORIAL

Descreva o ciclo de vida de embeddings em um banco vetorial.

Um *embedding* é a representação de um conteúdo — texto, imagem, áudio ou registro estruturado — como um vetor numérico em um espaço de alta dimensionalidade, no qual a proximidade geométrica aproxima itens semanticamente semelhantes. O banco vetorial é o repositório que armazena esses vetores junto de seus metadados e oferece busca por similaridade. Seu ciclo de vida começa na ingestão do objeto original e em sua fragmentação em unidades de tamanho adequado, etapa determinante da qualidade final, pois fragmentos longos diluem o significado e fragmentos curtos perdem contexto.

Na sequência, cada fragmento é submetido a um modelo de codificação que produz o vetor correspondente. Registra-se, nesse momento, a versão do modelo utilizado, informação indispensável porque vetores gerados por modelos diferentes não são comparáveis entre si. O armazenamento reúne o vetor, os metadados de filtragem — origem, data, autor, permissões, versão — e a referência ao objeto original, que permanece em seu repositório de origem. Segue-se a indexação por meio de algoritmos de busca aproximada, cuja configuração estabelece um compromisso explícito entre precisão da recuperação, latência e consumo de memória, associada à escolha da métrica de distância compatível com o modelo empregado.

A fase de uso consiste na consulta por vizinhos mais próximos, na qual a pergunta do usuário é convertida em vetor pelo mesmo modelo e comparada ao índice, normalmente com filtros por metadados que restringem o universo de busca e garantem o controle de acesso. O ciclo se completa com avaliação e manutenção: medem-se indicadores de qualidade da recuperação, monitoram-se latência e taxa de acerto e reindexa-se o acervo sempre que o modelo é atualizado, o que caracteriza uma operação custosa e que precisa ser planejada. Acrescentam-se ainda exigências de governança, como a exclusão de vetores derivados de dados removidos por solicitação do titular, requisito diretamente associado à legislação de proteção de dados. Em uma aplicação corporativa de geração aumentada por recuperação, esse ciclo é o que mantém as respostas ancoradas em documentos atualizados e autorizados.

9 QUESTÃO 8 — RISCOS DE OPERAÇÃO EM BANCOS NOSQL DISTRIBUÍDOS

Análise riscos de operação em bancos NoSQL distribuídos.

Operar um banco NoSQL distribuído significa administrar, de forma permanente, os compromissos descritos pelo teorema CAP. Diante de uma partição de rede, o sistema precisa optar entre preservar a consistência ou permanecer disponível, e essa escolha, feita em tempo de projeto por meio de níveis de consistência e quóruns, tem consequências operacionais concretas (KLEPPMANN, 2017). O primeiro risco, portanto, é a leitura de dados desatualizados quando se adota consistência eventual sem que o negócio tolere divergência — situação particularmente sensível em saldos, estoques e limites de crédito. O segundo é a percepção equivocada de que a replicação substitui o *backup*: réplicas propagam também a exclusão indevida e a corrupção lógica, de modo que apenas cópias íntegras e, sobretudo, restaurações efetivamente testadas asseguram a recuperação.

Um segundo conjunto de riscos decorre da distribuição física dos dados. Chaves de partição mal dimensionadas produzem concentração de carga e partições desbalanceadas; a entrada ou a saída de nós desencadeia rebalanceamento e reparação, processos que competem por rede e disco com a carga produtiva; e rotinas internas de compactação e de coleta de lixo podem introduzir picos de latência perceptíveis pelo usuário. Sob o aspecto econômico, modelos cobrados por capacidade ou por unidade de requisição transformam erros de modelagem em custo recorrente, e o crescimento silencioso de índices e de dados sem política de retenção agrava o problema. No plano da segurança, instâncias expostas sem autenticação, cifragem ausente e ausência de segregação de perfis constituem falhas documentadas em incidentes públicos e agravadas por exigências regulatórias, entre elas as da Lei Geral de Proteção de Dados.

Por fim, há riscos organizacionais frequentemente subestimados. A operação de sistemas com estado exige competências específicas, especialmente quando executada sobre orquestradores de contêineres, e a adoção de várias tecnologias distintas multiplica a superfície de conhecimento necessária. Painéis sem limiares definidos, alertas sem responsável designado e ausência de procedimentos de resposta documentados produzem a ilusão de observabilidade sem capacidade real de reação (MARCUSO, 2026). Mitigar esse conjunto de riscos exige tratar

disponibilidade, recuperação, segurança e custo como requisitos, definidos na mesma etapa em que se definem volume, latência e padrão de acesso.

10 QUESTÃO 9 — ESTRATÉGIA DE MONITORAMENTO PARA UM BANCO DE DOCUMENTOS

Proponha uma estratégia de monitoramento para um NoSQL à sua escolha.

A estratégia proposta a seguir tem como objeto um banco de documentos MongoDB, implantado em conjunto de réplicas e responsável pelo catálogo de uma aplicação de comércio eletrônico. Seu princípio organizador é a construção de camadas de observação que partem da experiência do usuário e descem até a infraestrutura, de modo que cada indicador técnico possa ser interpretado à luz de um efeito perceptível pelo negócio. Na camada superior definem-se objetivos de nível de serviço: latência das consultas críticas medida em percentis elevados, taxa de erro por operação e disponibilidade do conjunto de réplicas em janela mensal.

Na camada do banco monitoram-se as operações por segundo segregadas por tipo, o tempo de resposta por coleção, o atraso de replicação entre o nó primário e os secundários, a taxa de acerto do cache do mecanismo de armazenamento, o número de conexões ativas em relação ao limite configurado, a fila de espera por recursos e a ocorrência de varreduras completas de coleção, indício direto de índice ausente ou inadequado. O criador de perfis de consultas lentas complementa essas métricas ao identificar quais operações específicas degradam a experiência. Na camada de infraestrutura acompanham-se utilização de processador, memória residente, latência e operações de entrada e saída do disco, espaço livre e saturação de rede. À parte, verificam-se continuamente a execução dos backups, o tempo decorrido desde a última cópia válida e o resultado dos testes periódicos de restauração, confrontados com os objetivos de ponto e de tempo de recuperação acordados.

A estratégia se completa com a disciplina operacional. Cada alerta possui limiar justificado, responsável identificado e procedimento de resposta documentado, evitando notificações que não conduzem a ação. Os painéis são organizados por público — um painel executivo com os objetivos de nível de serviço e painéis técnicos por componente — e os registros de consultas lentas alimentam uma

revisão periódica de índices e de modelagem. Recomenda-se, ainda, ensaio trimestral de restauração e simulação de falha do nó primário, pois somente o exercício efetivo comprova que o plano de recuperação funciona.

11 QUESTÃO 10 — ARQUITETURA POLYGLOT PARA UM MARKETPLACE

Construa uma arquitetura polyglot para um marketplace e justifique cada banco.

A persistência poliglota consiste em empregar tecnologias de armazenamento distintas dentro de uma mesma solução, atribuindo a cada uma o padrão de acesso que ela resolve melhor (SADALAGE; FOWLER, 2012). Um marketplace é caso exemplar dessa necessidade, pois reúne, em um único produto, transações financeiras que exigem garantias fortes, catálogo heterogêneo mantido por milhares de vendedores, busca textual com filtros, alto volume de eventos de comportamento, análise de relacionamentos para prevenção a fraudes e recomendação por similaridade. O Quadro 3 apresenta a arquitetura proposta.

Quadro 3 — Arquitetura poliglota proposta para um marketplace

Componente	Tecnologia	Justificativa
Pedidos, pagamentos e repasses	PostgreSQL	Transações com garantias ACID, integridade referencial e conciliação financeira auditável
Catálogo e anúncios	MongoDB	Atributos variáveis por categoria e evolução frequente do conteúdo, com leitura por agregado
Sessão, carrinho e limitação de requisições	Redis	Latência mínima, expiração automática e estruturas próprias para contadores e rankings
Busca e navegação por filtros	OpenSearch	Índice invertido, relevância, sinônimos e filtros facetados sobre o catálogo
Eventos de comportamento e telemetria	Apache Cassandra	Escrita distribuída em grande volume, particionada por usuário e por intervalo de tempo
Antifraude e recomendação por relações	Neo4j	Consultas de caminho entre contas, dispositivos e endereços, com profundidade variável
Busca semântica e itens similares	Banco vetorial	Recuperação por similaridade de embeddings, com filtros por metadados

Componente	Tecnologia	Justificativa
Mídias e arquivos	Armazenamento de objetos	Custo por gigabyte reduzido, distribuição por rede de entrega de conteúdo e durabilidade elevada

Fonte: elaborado pelo autor com base em Sadalage e Fowler (2012) e em Marcusso (2026).

A coerência entre esses repositórios exige disciplina arquitetural. O banco relacional é definido como fonte da verdade para pedidos e pagamentos, e as demais bases são alimentadas por eventos publicados a partir das mudanças de estado, mecanismo que estabelece consistência eventual controlada entre catálogo, índice de busca e vetores. Cada projeção precisa ser reconstruível a partir da fonte, de modo que uma falha de sincronização se resolva por reprocessamento e não por intervenção manual.

Cabe ressaltar o custo dessa decisão. Cada tecnologia adicional acrescenta operação, monitoramento, backup, segurança, licenciamento e necessidade de competência específica na equipe, razão pela qual a arquitetura descrita se justifica em um marketplace de porte, mas seria desproporcional em um projeto inicial. Em estágios iniciais, a recomendação é concentrar a solução em um banco relacional acrescido de um cache, incorporando cada tecnologia especializada apenas quando um padrão de acesso concreto demonstrar que a alternativa existente não atende ao requisito.

12 CONSIDERAÇÕES FINAIS

O percurso pelas dez questões confirma a tese central do material da disciplina: NoSQL não designa um banco de dados, mas uma família de escolhas técnicas, cada qual concebida para resolver melhor do que as alternativas um determinado padrão de acesso (MARCUSO, 2026). Documentos favorecem agregados de conteúdo variável; estruturas chave-valor privilegiam latência mínima; bancos wide-column absorvem escrita distribuída em larga escala; grafos tornam eficiente a travessia de relacionamentos; séries temporais organizam medições ao longo do tempo; índices invertidos viabilizam busca textual relevante; e bancos vetoriais sustentam a recuperação por similaridade que fundamenta as aplicações de inteligência artificial.

Disso decorre que a decisão tecnológica é, antes de tudo, uma decisão de requisitos. Volume, latência alvo, exigência de consistência, cardinalidade da chave, política de retenção, distribuição geográfica, custo por unidade de capacidade, obrigações regulatórias e maturidade da equipe formam o conjunto de critérios que precede qualquer preferência por produto. Mais importante, a operação — replicação, backup testado, segurança, observabilidade com limiares e procedimentos de resposta — deve ser projetada desde o início, e não acrescentada após o primeiro incidente. Escolher a tecnologia pelo problema e projetar a operação junto com o modelo constitui, portanto, a síntese prática de todo o ciclo estudado.

REFERÊNCIAS

ELMASRI, Ramez; NAVATHE, Shamkant B. Sistemas de banco de dados. 6. ed. São Paulo: Pearson Addison Wesley, 2011.

INFLUXDATA. InfluxDB documentation. Disponível em: <https://docs.influxdata.com/>. Acesso em: 2 set. 2026.

KLEPPMANN, Martin. Designing data-intensive applications: the big ideas behind reliable, scalable, and maintainable systems. Sebastopol: O'Reilly Media, 2017.

MARCUSSO, Nivaldo T. Bancos de dados NoSQL: ciclo completo. Material didático da disciplina Tratamento e Armazenamento da Informação. Campinas: FACAMP, 2026.

REDMOND, Eric; WILSON, Jim R. Seven databases in seven weeks: a guide to modern databases and the NoSQL movement. Dallas: Pragmatic Bookshelf, 2012.

ROBINSON, Ian; WEBBER, Jim; EIFREM, Emil. Graph databases: new opportunities for connected data. 2. ed. Sebastopol: O'Reilly Media, 2015.

SADALAGE, Pramod J.; FOWLER, Martin. NoSQL distilled: a brief guide to the emerging world of polyglot persistence. Upper Saddle River: Addison-Wesley, 2012.

SILBERSCHATZ, Abraham; KORTH, Henry F.; SUDARSHAN, S. Sistema de banco de dados. 6. ed. Rio de Janeiro: Elsevier, 2012.

TIMESCALE. TimescaleDB documentation. Disponível em: <https://docs.timescale.com/>. Acesso em: 2 set. 2026.

APÊNDICE A — PROMPT UTILIZADO

Reproduz-se a seguir, na íntegra, o prompt utilizado na elaboração deste trabalho com auxílio de inteligência artificial, conforme orientação do professor da disciplina.

PROMPT — QUESTÕES DISSERTATIVAS DE NOSQL (ENTREGA ABNT)

CONTEXTO

Você é um assistente acadêmico especializado em bancos de dados. Estou cursando a disciplina Tratamento e Armazenamento da Informação na FACAMP (2026), com o Prof. Me. Nivaldo T. Marcusso. A aula em questão trata de Bancos de Dados NoSQL — ciclo completo (requisitos, modelagem, criação, armazenamento, hospedagem e monitoramento), cobrindo os modelos: document, key-value, wide-column, graph, time-series, search, vector e multi-model. Preciso entregar as 10 questões dissertativas do slide 63 da apresentação, respondidas e formatadas em um Google Docs no padrão ABNT.

TAREFA

Responda, de forma dissertativa e fundamentada, às 10 questões abaixo e monte o documento final formatado em ABNT.

1. Explique por que a modelagem NoSQL costuma ser orientada aos padrões de consulta.
2. Compare document e key-value em estrutura, consulta e casos de uso.
3. Mostre como uma partition key inadequada prejudica Cassandra ou DynamoDB.
4. Explique quando um grafo é superior a joins relacionais.
5. Compare InfluxDB e TimescaleDB do ponto de vista conceitual.
6. Explique índice invertido e seu papel em Elasticsearch/OpenSearch.
7. Descreva o ciclo de vida de embeddings em um banco vetorial.
8. Analise riscos de operação em bancos NoSQL distribuídos.
9. Proponha uma estratégia de monitoramento para um NoSQL à sua escolha.
10. Construa uma arquitetura polyglot para um marketplace e justifique cada banco.

COMO RESPONDER

- Extensão: 2 a 3 parágrafos por questão (aproximadamente 200 a 350 palavras cada).
- Estrutura de cada resposta: (a) definição/conceito central; (b) desenvolvimento técnico com trade-offs; (c) exemplo prático concreto de mercado ou de aplicação real.
- Linguagem: impessoal, acadêmica, em português do Brasil, sem bullet points dentro das respostas (texto corrido dissertativo). Tabelas são permitidas apenas nas questões comparativas (2, 5 e 10), sempre acompanhadas de texto explicativo.
- Consistência com a aula: usar os conceitos apresentados nos slides — teorema CAP, consistência eventual x forte, modelagem query-first, denormalização consciente,

particionamento e hotspots, índices seletivos, TTL/retenção, observabilidade e SLOs, polyglot persistence.

- Termos técnicos em inglês (partition key, hotspot, index, embedding, throughput) podem ser mantidos em *itálico* na primeira ocorrência.
- Nada de invenção: não citar fontes, autores, páginas ou números sem certeza de que existem. Se um dado for aproximado, indicar isso no texto.

REFERÊNCIAS

Os slides da aula disponibilizados e a bibliografia abaixo, citada ao longo do texto no formato autor-data ABNT (AUTOR, ano):

- KLEPPMANN, Martin. Designing Data-Intensive Applications.
- SADALAGE, Pramod J.; FOWLER, Martin. NoSQL Distilled.
- REDMOND, Eric; WILSON, Jim. Seven Databases in Seven Weeks.
- ELMASRI, Ramez; NAVATHE, Shamkant B. Sistemas de Banco de Dados.
- SILBERSCHATZ, Abraham; KORTH, Henry F.; SUDARSHAN, S. Sistema de Banco de Dados.
- Documentação oficial dos produtos citados (MongoDB, Redis, Cassandra, DynamoDB, Neo4j, InfluxDB, TimescaleDB, Elasticsearch/OpenSearch, Milvus/Qdrant/Weaviate e provedores de nuvem).

Usar no mínimo 4 fontes distintas ao longo do trabalho e listar todas — e apenas as efetivamente citadas — na seção de Referências.

FORMATAÇÃO ABNT DO DOCUMENTO

- Capa: FACAMP — Faculdade de Campinas; curso; disciplina "Tratamento e Armazenamento da Informação"; título "Questões Dissertativas — Bancos de Dados NoSQL"; nome do aluno: Felipe Furlan; professor: Prof. Me. Nivaldo T. Marcusso; Campinas; 2026.
- Fonte Arial 12, espaçamento entre linhas 1,5, texto justificado, recuo de primeira linha de 1,25 cm.
- Margens: 3 cm superior e esquerda; 2 cm inferior e direita.
- Numeração de páginas no canto superior direito.
- Seções numeradas: cada questão vira uma seção (1 INTRODUÇÃO, 2 QUESTÃO 1 — título curto do tema, ..., 12 CONSIDERAÇÕES FINAIS, REFERÊNCIAS).
- Enunciado de cada questão reproduzido em *itálico* logo abaixo do título da seção, antes da resposta.
- Citações diretas curtas entre aspas com (AUTOR, ano, p. X); citações longas (mais de 3 linhas) com recuo de 4 cm, fonte 10, espaçamento simples, sem aspas.
- Referências ao final, em ordem alfabética, alinhadas à esquerda, espaçamento simples e separadas por uma linha em branco.
- Incluir uma Introdução curta (1 parágrafo) e Considerações finais (1 a 2 parágrafos amarrando o conceito de escolha por padrão de acesso).

- Ao final do documento, acrescentar um Apêndice A — Prompt utilizado, reproduzindo integralmente este prompt (exigência do professor).

ENTREGA

Gerar o resultado como um Google Docs no meu Google Drive, com o nome do arquivo: TAI

- Questões Dissertativas NoSQL - Felipe Furlan. Ao terminar, enviar o link e um resumo de uma linha do que foi produzido.